PYTHON LISTS

A practice workbook built around what you have learned so far.

50 QUESTIONS

15 Easy | 15 Medium | 15 Hard | 5 Complex Reasoning

YOUR CURRENT TOPIC SCOPE

- List methods: append(), insert(), remove(), pop(), sort(), reverse()
- String methods: capitalize(), replace(), find(), count(), endswith()
- Conditionals: if / else, plus basic variables, comparisons and print()
- No for/while loops, no list traversal, and no advanced methods.

Write code for every question. The workbook contains questions, not solutions.

How to use this workbook

Write the full Python code, not only the final answer. After each question, run your code and check the output. In this version, challenge comes from choosing and sequencing the methods you already know - not from using topics you have not studied yet.

- Do not use: `for`, `while`, list comprehensions, `len()`, `max()`, `min()`, `extend()`, slicing, or list traversal.
- Small but important details: `remove(value)` removes the first matching item; `pop()` removes and returns an item; `find()` returns `-1` if text is not found; `endswith()` is case-sensitive; `capitalize()` uppercases only the first character and lowercases the rest.
- Difficulty rule: Hard and Complex questions are intentionally multi-step, but every operation remains inside the skills listed on the cover.

EASY | Questions 1-15

Direct practice with the list methods you have already studied. Focus on writing the exact operation and predicting the final list.

Q01 Add a new score

A quiz app stores three scores. Add one new score to the end of the list.

```
scores = [18, 24, 31]
```

Your task: Use `append()` to add `27`, then print `scores`.

Practice: append()

Q02 Insert a city in a specific position

A travel list is missing Mathura between Delhi and Jaipur.

```
cities = ["Delhi", "Agra", "Jaipur"]
```

Your task: Use `insert()` to put `"Mathura"` at index `1`. Print the updated list.

Practice: insert()

Q03 Remove one matching item

The snack list contains `"fruit"` twice. You only need to remove the first matching occurrence.

```
snacks = ["chips", "fruit", "biscuits", "fruit"]
```

Your task: Use `remove("fruit")` and print the result.

Practice: remove() removes the first matching item

Q04 Take the last item out

A playlist has one track at the end that should be taken out and stored separately.

```
playlist = ["Intro", "Song A", "Song B"]
```

Your task: Use `pop()` to store the removed track in `removed_track`. Print both variables.

Practice: pop()

Q05 Take the first order out of a queue

The first order is ready to be handled. Remove just that first item.

```
orders = ["Order-101", "Order-102", "Order-103"]
```

Your task: Use `pop(0)` to store the first order in `current_order`. Print the order and the remaining list.

Practice: pop(index)

Q06 Sort numbers from low to high

A list of item prices has arrived in a mixed order.

```
prices = [430, 125, 300, 210]
```

Your task: Use `sort()` to arrange the prices in ascending order. Print the list.

Practice: sort()

Q07 Sort names alphabetically

A small guest list needs to be displayed in alphabetical order.

```
names = ["Zoya", "Aman", "Riya", "Kabir"]
```

Your task: Use `sort()` and print the updated list.

Practice: sort() on strings

Q08 Reverse without sorting

The list order itself must be flipped. Do not alphabetize or numerically sort it first.

```
numbers = [4, 11, 2, 9]
```

Your task: Use `reverse()` and print the list.

Practice: reverse()

Q09 Arrange in descending numeric order

The numbers should end up from highest to lowest, using only methods you have learned.

```
numbers = [31, 7, 18, 12]
```

Your task: Call `sort()` and then `reverse()`. Print the final list.

Practice: sort() + reverse()

Q10 Add two supplies

A stationery list needs an eraser added at the end and a marker placed between the existing items.

```
supplies = ["notebook", "pen"]
```

Your task: Use `append()` to add `"eraser"`, then `insert()` to place `"marker"` at index `1`. Print the list.

Practice: append() + insert()

Q11 Replace one list entry by two operations

Remove `"blue"` from the color list, then add `"black"` at the end.

```
colors_list = ["red", "blue", "green"]
```

Your task: Use `remove()` and `append()`, then print the result.

Practice: remove() + append()

Q12 Remove an item from the middle

The second ticket is cancelled. Save its value before displaying the remaining tickets.

```
tickets = ["Ticket-A", "Ticket-B", "Ticket-C"]
```

Your task: Use `pop(1)` to store the removed ticket in `cancelled_ticket`. Print both the variable and list.

Practice: pop(index)

Q13 Notice what remove() actually removes

There are two copies of the number `8`. You have learned `remove()`, not a method that removes all matches.

```
values = [8, 4, 8, 2]
```

Your task: Call `values.remove(8)`, print the list, and write one sentence explaining how many `8`s remain.

Practice: remove() with duplicates

Q14 Swap the last supply

Remove the last supply from the list, then add `"scale"` to the end.

```
supplies = ["pen", "pencil", "eraser"]
```

Your task: Use `pop()` and `append()`. Print the updated list and the item returned by `pop()`.

Practice: pop() + append()

Q15 Put an urgent task first

An urgent call must appear before the other tasks.

```
tasks = ["Email", "Meeting", "Report"]
```

Your task: Use `insert(0, "Urgent call")` and print the updated list.

Practice: insert() at index 0

MEDIUM | Questions 16-30

Combine two or three known operations. Some questions add `if/else` or one of the string methods you listed. No loops are needed.

Q16 Update and alphabetize the supply list

The pencil is out of stock and a scale has arrived. Update the list and display it alphabetically.

```
items = ["pen", "pencil", "eraser"]
```

Your task: Remove `"pencil"`, append `"scale"`, call `sort()`, and print `items`.

Practice: remove() + append() + sort()

Q17 Find the top score

Sort the scores high-to-low, then remove and store the highest score.

```
scores = [45, 72, 60]
```

Your task: Add `81`, call `sort()` then `reverse()`, and use `pop(0)` to store the top score. Print the score and remaining list.

Practice: append() + sort() + reverse() + pop()

Q18 Serve one waiting ticket

Only one ticket is being handled now. A new ticket also arrives.

```
waiting = ["A12", "A13", "A14"]
```

Your task: Insert `"A11"` at index `0`, pop index `0` into `current_ticket`, then append `"A15"`. Print both.

Practice: insert() + pop() + append()

Q19 Place a number, then show high-to-low

The number `6` must first be inserted at index `1`. After that, show the list from high to low.

```
seats = [8, 3, 5]
```

Your task: Use `insert(1, 6)`, then `sort()` and `reverse()`. Print `seats`.

Practice: insert() + sort() + reverse()

Q20 Remove only one duplicate from a cart

The cart accidentally has two copies of `"Pen"`. Remove one copy only, then add a pencil.

```
cart = ["Book", "Pen", "Pen", "Ruler"]
```

Your task: Use `remove("Pen")` and `append("Pencil")`. Print the cart and state how many `"Pen"` entries remain.

Practice: remove() + append()

Q21 Choose a temperature label

A simple weather report stores its result in a list. Use the threshold of 30 degrees.

```
temperature = 34
labels = []
```

Your task: If `temperature >= 30`, append `"Hot"` to `labels`; otherwise append `"Normal"`. Print `labels`.

Practice: if/else + append()

Q22 Clean a filename and route it

Remove the suffix `_final` from the filename. If the cleaned filename ends with uppercase `.PDF`, put it in `upload_files`; otherwise put it in `manual_check`.

```
file_name = "report_final.PDF"
clean_file = file_name.replace("_final", "")
upload_files = []
manual_check = []
```

Your task: Write the `if/else` using `endswith()` and append the cleaned filename to the correct list. Print both lists.

Practice: replace() + endswith() + if/else

Q23 Normalize one username

`capitalize()` makes the first character uppercase and the remaining characters lowercase. Use that exact behavior.

```
username = "sHREYASH"
members = ["Aditi", "Rahul"]
```

Your task: Store `username.capitalize()` in `clean_username`, insert it at index `1` in `members`, and print the list.

Practice: capitalize() + insert()

Q24 Flag a repeated word

Count how often the substring `"error"` appears in the message. Use that count to choose one list.

```
message = "error: payment error"
flagged = []
approved = []
```

Your task: If `message.count("error") >= 2`, append the message to `flagged`; otherwise append it to `approved`. Print both lists.

Practice: string count() + if/else + append()

Q25 Check whether an email contains @

This is a basic practice check, not full email validation. `find()` returns `-1` when the substring is not found.

```
email = "help@shop.in"
valid_contacts = []
review_contacts = []
```

Your task: Use `email.find("@") != -1` in an `if/else`. Append the email to the matching list and print both.

Practice: find() + if/else + append()

Q26 Choose an order action

The payment status is stored as text. Decide which action should be added to the action list.

```
payment_status = "order_paid"
actions = []
```

Your task: If `payment_status.endswith("paid")`, append `"Pack order"`; otherwise append `"Hold order"`. Print `actions`.

Practice: endswith() + if/else

Q27 Count a character in a word

Use the string method `count()` to count `"a"` in the word. Choose a message based on the result.

```
word = "banana"
checks = []
```

Your task: If `word.count("a") > 2`, append `"Repeated a"`; otherwise append `"Normal count"`. Print `checks`.

Practice: count() + if/else

Q28 Rename one image file

Replace `_old` with `_new`. Route the resulting filename based on whether it ends with `.png`.

```
file_name = "profile_old.png"
renamed = file_name.replace("_old", "_new")
image_files = []
other_files = []
```

Your task: Write the `if/else` using `renamed.endswith(".png")`, append to the appropriate list, and print both.

Practice: replace() + endswith() + append()

Q29 Use find() to prioritize a ticket

The message may or may not mention `server`. The ticket list already contains two tickets.

```
message = "urgent: urgent: server"
queue = ["Ticket-2", "Ticket-3"]
```

Your task: If `message.find("server") != -1`, insert `"Check server log"` at index `0`; otherwise append `"General review"`. Print `queue`.

Practice: find() + if/else + insert()/append()

Q30 Normalize a name before inserting

Clean the name using `capitalize()`. Then put the cleaned name first if it ends with `"a"`; otherwise add it to the end.

```
name = "nEHA"
clean_name = name.capitalize()
new_members = ["Ravi", "Zoya"]
```

Your task: Write the `if/else` using `clean_name.endswith("a")` and update `new_members`. Print the result.

Practice: capitalize() + endswith() + insert()/append()

HARD | Questions 31-45

These are harder because you must plan a sequence of known operations and track what changes. They still use only the topics you listed - no `for` or `while` loops.

Q31 Prioritize and dispatch one order

Sort the existing codes high-to-low, put the priority order at the front, remove the first code as the one being dispatched, then label the dispatch.

```
orders = ["ORD-104", "ORD-101", "ORD-103"]
actions = []
```

Your task: Call `sort()` then `reverse()`, insert `"ORD-100"` at index `0`, and store `pop(0)` in `next_order`. If it ends with `"100"`, append `"Priority dispatch"` to `actions`, else append `"Standard dispatch"`. Print all variables.

Practice: insert() + sort() + reverse() + pop() + endswith()

Q32 Review the lowest mark

Add a new mark, sort from low to high, and remove the lowest mark for a separate review. Decide whether that mark needs review.

```
marks = [58, 76, 64]
marks_to_review = []
status = []
```

Your task: Append `81`, sort `marks`, and store `pop(0)` in `lowest_mark`. If it is below `60`, append `"Review required"` to `status`; otherwise append `"Ready"`. Print the results.

Practice: append() + sort() + pop() + if/else

Q33 Rebuild a cart in a new order

Remove the mug, place a pencil at index `1`, add a ruler, and then reverse the final cart.

```
cart = ["Notebook", "Pen", "Mug"]
```

Your task: Use `remove()`, `insert()`, `append()`, and `reverse()` in the correct sequence. Print the final cart after each operation.

Practice: four list operations in sequence

Q34 Route a cleaned invoice filename

Clean the filename first. If it contains `"invoice"` and ends with `.csv`, put it at the start of the processing queue. Otherwise, place it in manual review. Use nested `if` statements.

```
file_name = "invoice_final.csv"
cleaned = file_name.replace("_final", "")
processing_queue = ["report.csv"]
manual_review = []
```

Your task: Use `find()` in the outer `if` and `endswith()` in the inner `if`. Insert the cleaned filename at index `0` only when both checks pass; otherwise append it to `manual_review`. Print both lists.

Practice: replace() + nested if/else + find() + endswith()

Q35 Decide whether a post needs moderation

A post is flagged only when `"WIN"` occurs at least twice and `"prize"` can be found. Route the message to one of two lists.

```
post = "WIN a prize WIN today"
flagged_posts = []
approved_posts = []
```

Your task: Use `post.count("WIN")` and `post.find("prize")` with nested `if/else` statements. Append the post to the correct list and print both.

Practice: count() + find() + nested if/else

Q36 Build an error-check action list

The log may mention a disk issue and may repeat the word `warning`. Add the relevant actions without using any loop.

```
log = "warning: disk warning"
checks = []
```

Your task: If `log.find("disk") != -1`, insert `"Disk check"` at index `0`; otherwise append `"General check"`. Then, if `log.count("warning") >= 2`, append `"Repeated warning"`; otherwise append `"Single warning"`. Print `checks`.

Practice: find() + count() + list insertion/appending

Q37 Prioritize a ticket from sorted codes

Sort codes in reverse order, add a new priority code at the front, then take the first ticket for handling. Decide the handling label.

```
tickets = ["T-13", "T-10", "T-11"]
actions = []
```

Your task: Call `sort()` then `reverse()`, insert `"T-20"` at index `0`, and store `pop(0)` in `current_ticket`. If it ends with `"20"`, append `"Priority ticket"`; otherwise append `"Standard ticket"`. Print the results.

Practice: sort() + reverse() + insert() + pop() + endswith()

Q38 Clean and update an attendee list

Normalize one name. Add it to the attendee list, remove a cancelled attendee, and sort the final list.

```
name = "aLEX mORGAN"
attendees = ["Mia Chen", "Noah Patel"]
```

Your task: Use `capitalize()` to create `clean_name`; insert it at index `1`, remove `"Noah Patel"`, sort the list, and print it. Remember that `capitalize()` does not capitalize every word.

Practice: capitalize() + insert() + remove() + sort()

Q39 Separate the highest and lowest counts

Sort counts high-to-low. Store the highest and lowest values separately, then produce a stock alert based on the lowest value.

```
counts = [8, 3, 11, 6]
alerts = []
```

Your task: Call `sort()` then `reverse()`. Store `pop(0)` in `largest` and `pop()` in `smallest`. If `smallest < 5`, append `"Restock"` to `alerts`; otherwise append `"Stock okay"`. Print all variables.

Practice: sort() + reverse() + pop() + if/else

Q40 Route a repeated payment issue

Replace `failed` with `needs review`. If `payment` appears at least twice, put the cleaned text at the front of the issue queue; otherwise append it.

```
message = "payment failed payment failed"
cleaned_message = message.replace("failed", "needs review")
issue_queue = ["Case-22", "Case-23"]
```

Your task: Use `count()` with `if/else` to update `issue_queue`. Also create `location_note`: if `message.find("payment") == 0`, set it to `"Payment queue"`; otherwise set it to `"General queue"`. Print both.

Practice: replace() + count() + find() + insert()/append()

Q41 Approve or reject a renamed image

Rename `_old` to `_final`. Only filenames ending in `.png` go into the approved list. Sort the approved list and reverse it for display.

```
file_name = "photo_old.png"
cleaned = file_name.replace("_old", "_final")
approved = []
rejected = []
```

Your task: Use `endswith()` and `if/else` to append the cleaned name to the right list. Then sort and reverse `approved`. Print both lists.

Practice: replace() + endswith() + if/else + sort()/reverse()

Q42 Choose the event check-in action

The code may end in `confirmed`. Put a priority check-in instruction first for a confirmed event; otherwise add a standard instruction at the end.

```
event_code = "VIP-confirmed"
checkin_actions = ["Open registration", "Verify ID"]
```

Your task: Use `endswith("confirmed")` and `if/else` to insert `"VIP check-in"` at index `0` or append `"Standard check-in"`. Then print the action list.

Practice: endswith() + insert()/append()

Q43 Update a cancelled reservation list

Remove the cancelled room, add a replacement room at index `1`, sort the list, then take the first room code out for confirmation.

```
reservations = ["Room-2", "Room-4", "Room-3", "Room-1"]
```

Your task: Remove `"Room-4"`, insert `"Room-5"` at index `1`, sort the list, and store `pop(0)` in `confirmed_room`. Print both the room and remaining list.

Practice: remove() + insert() + sort() + pop()

Q44 Turn a defect note into a review task

Replace `defect` with `review`. If the original note contains `crack`, place the cleaned note at the top of the review queue; otherwise append a general review message.

```
quality_note = "defect: crack found"
review_queue = ["Check packaging"]
```

Your task: Create `clean_note` with `replace()`. Use `find()` and `if/else` to update `review_queue`, then print the cleaned note and queue.

Practice: replace() + find() + insert()/append()

Q45 Prepare a release package

Remove `_final` from a package filename. If the build status ends with `approved`, put the filename at the front of the release list; otherwise add the filename to the hold list.

```
package_name = "app_v2_final.zip"
build_status = "build_approved"
release_files = ["manual.pdf"]
hold_files = []
```

Your task: Create `clean_name` with `replace()`. Use `endswith()` and `if/else` to update the correct list. Sort and reverse `release_files`, then print both lists.

Practice: replace() + endswith() + if/else + sort()/reverse()

COMPLEX REASONING | Questions 46-50

Small real-world scenarios that combine multiple learned ideas. “Complex” here means careful multi-step reasoning within your current syllabus - not advanced algorithms.

Q46 Event check-in with a cancelled attendee

A new attendee name needs cleaning. The event code decides whether the person is placed first or last. One existing attendee has cancelled and must be removed.

```
name = "mAYA SINGH"
event_code = "TECH-PASS"
attendees = ["Aarav", "Isha"]
```

Your task: Create `clean_name` using `capitalize()`. If `event_code.endswith("PASS")`, insert the name at index `0`; otherwise append it. Remove `"Isha"`, sort `attendees`, and print it. Also explain the exact result of `capitalize()` for the provided name.

Practice: capitalize() + endswith() + insert()/append() + remove() + sort()

Q47 Online order dispatch decision

The customer has paid only if the status ends in `paid`. The mug was removed from the order and a pencil was added. The cart must be sorted before the dispatch decision is recorded.

```
payment_status = "order_paid"
cart = ["Notebook", "Mug", "Pen"]
dispatch_queue = []
actions = []
```

Your task: Remove `"Mug"`, append `"Pencil"`, then sort `cart`. If `payment_status.endswith("paid")`, insert `"ORDER-104"` at the front of `dispatch_queue` and append `"Dispatch approved"` to `actions`; otherwise append `"HOLD-104"` and `"Payment hold"`. Print all lists.

Practice: remove() + append() + sort() + endswith() + if/else

Q48 Moderate a promotional message

A message should be reviewed when `WIN` appears at least twice. If that condition passes, check whether `phone` appears too. The cleaned message replaces every occurrence of `WIN` with `promotion`.

```
message = "WIN a phone WIN now"
cleaned_message = message.replace("WIN", "promotion")
review_queue = ["Case-201", "Case-202"]
approved_queue = []
```

Your task: Use nested `if/else` statements with `count()` and `find()`. When both conditions pass, insert the cleaned message at index `0`; if the count passes but `phone` is absent, append it to `review_queue`; if the count fails, append it to `approved_queue`. Print all variables.

Practice: replace() + count() + find() + nested if/else

Q49 Prepare a contest score summary

Add the latest score, order the scores high-to-low, then separate the highest and lowest values. Create alerts for a top score of at least 90 and a lowest score below 75.

```
scores = [72, 95, 81]
alerts = []
```

Your task: Append `88`, sort and reverse `scores`, store `pop(0)` in `highest` and `pop()` in `lowest`. If `highest >= 90`, append `"Top score achieved"`; otherwise append `"Top score below target"`. If `lowest < 75`, append `"Lowest score needs review"`; otherwise append `"Lowest score acceptable"`. Print `highest`, `lowest`, the remaining scores, and `alerts`.

Practice: append() + sort() + reverse() + pop() + if/else

Q50 Handle a return with a missing receipt

The customer message repeats `wrong size` twice and the receipt status ends with `missing`. Route this case to manual review. Also remove one cancelled return from the existing queue and add a new case.

```
reason = "wrong size; wrong size"
receipt_status = "receipt_missing"
return_queue = ["R-102", "R-103"]
approved_returns = []
manual_review = []
```

Your task: If `reason.count("wrong size") >= 2`, check `receipt_status.endswith("missing")`: when true, insert `"R-101"` at index `0` of `manual_review`; otherwise append it to `approved_returns`. If the count is lower, append it to `approved_returns`. Then remove `"R-103"` from `return_queue`, append `"R-104"`, sort the queue, and print all three lists.

Practice: count() + endswith() + nested if/else + list methods

What You Will Learn

This workbook is designed to strengthen your current foundation before introducing loops or algorithm patterns.

- Use list methods deliberately. Choose between `append()`, `insert()`, `remove()`, `pop()`, `sort()`, and `reverse()` based on the required change.
- Predict list state after each step. Track how a list changes when you apply multiple operations in a particular order.
- Understand duplicates and removal. Recognize that `remove()` removes only the first matching item, while `pop()` removes an item by position and returns it.
- Combine lists with conditions. Use `if/else` to choose which list to update or which action to record.
- Apply string methods to decisions. Use `capitalize()`, `replace()`, `find()`, `count()`, and `endswith()` to clean text, check simple conditions, and route items.
- Break small real-world tasks into steps. Practice order queues, event check-in, filename handling, moderation, scores, and returns without writing loops.
- Build confidence before learning loops. Strengthen your control over variables, method calls, and conditions first. Once these are comfortable, loops can be learned as a separate new topic.

Important: This is not meant to teach loops indirectly. It deliberately avoids them because you said you have not studied them yet. When you learn `for` and `while`, a new workbook can add traversal, duplicates across an entire list, and algorithmic patterns.

Recommended practice routine

Finish Easy independently. In Medium, explain why each `if/else` branch runs. In Hard and Complex, write the steps on paper first, then code them in order. Do not jump to loops to solve a question; ask for a revised version if a prompt appears to require one.